

ASSIGNMENT 4

Instructions for Students

- Create separate .js files for each question.
- Use descriptive variable names, indentation, and comments to explain logic.
- Use only core JavaScript (ES6+) — no frameworks.
- Test using Node.js console or browser console.

Q 1: E-Commerce Product Manager (Classes + Objects)

You're building an **E-commerce Admin Panel**. Create a **Product** class with properties like **id**, **name**, **price**, and **category**.

- Add methods to:
 - Apply a discount (e.g., 10%)
 - Display product details in a formatted string
 - Create multiple product objects and store them in an array.
 - Display products with price > 1000 in the console using a filter.
-

Q 2: Student Form Validator (Forms + RegExp)

Create a **student registration form** with fields: **Name**, **Email**, **Phone**, **Password**.
Use **JavaScript + RegExp** to validate:

- Name: only alphabets
- Email: valid format (**example@domain.com**)
- Phone: exactly 10 digits
- Password: must contain 1 uppercase, 1 number, and 1 special character

Show red border + error message if invalid, green if valid.

Q 3: Library Management System (Classes + Objects)

Design a **Book** class with properties like **title**, **author**, **ISBN**, **isIssued**.

- Create methods **issueBook()** and **returnBook()**.
 - Create an array of book objects.
 - Display all available books (not issued).
 - Allow a user to issue a book by searching ISBN.
-

Q 4: Custom Form Builder (Forms + Classes)

Build a dynamic **form generator** using a **FormBuilder** class that takes an array of field objects (e.g., **{type: 'text', label: 'Username'}**) and creates a form dynamically using **innerHTML**.

Note

- Add a method **getFormData()** that returns all entered values as an object when user clicks "Submit".
-

Q 5: Movie Ticket Booking (Objects + RegExp)

Build a movie ticket booking form for an app.
Validate user input:

- **Name** (alphabets only)
- **Email** (proper format)

- **Seats** (1 to 10 only)

After successful validation, store booking info in an object { **name**, **email**, **seats** } and display the ticket details.

Q 6: Employee Management System (Classes + Object Methods)

Create a class **Employee** with properties: **id**, **name**, **department**, **salary**.

- Add method **getAnnualSalary()** and **applyBonus(percent)**.
 - Create 5 employee objects and calculate annual salary for each.
 - Use **reduce()** to calculate total annual payout of the company.
-

Q 7: Login Form Validation using RegExp

Create a login form with username & password.
Use RegExp to check:

- Username: at least 5 characters
- Password: at least 8 characters, must include number, uppercase, lowercase, special character

If validation passes → show success message; else → show specific error messages.

Q 8: Dynamic Object Updater

Create an object **user** = { **name**: "John", **email**: "john@mail.com", **age**: 21 }.

Build a **form** that allows editing these values.

On submit, the form should **update the user object** in real time.

Show updated object details below the form using DOM.

Q 9: Shopping Cart Total (Classes + RegExp for Coupon)

Build a shopping cart system using a **Cart** class.

- Add items with **name**, **price**, **quantity**.
 - Calculate total using **getTotal()**.
 - Apply coupon validation using **RegExp**:
 - Coupon format must be like **SAVE20** or **DISC10**.
 - If valid, apply corresponding % discount and show final total.
-